

Every declaration on this sheet is marked in halves — the **TYPE** line scores separately from what follows it. So write the half you know. A blank line scores nothing; half a line scores half.

1. A city bicycle-hire scheme records every hire. [10]

The bikes available are of these models only: **Classic, Electric, Cargo, Folding, Tandem**.

Each hire records:

- a hire number, which may be any combination of letters and digits
- the customer's name
- the date of hire
- the model of bike used
- whether the bike has been returned

(a) Write pseudocode to declare the **enumerated** data type **BikeModel**. [2]

.....

(b) Write pseudocode to declare the **composite** data type **Hire**. Use the enumerated type from part (a). [4]

.....

.....

.....

(c) Write the pseudocode statement to set up a variable for **one** record of type **Hire**. [1]

.....

(d) Write pseudocode to store the following hire in the variable you set up in part (c). [3]

Field	Data
hire number	BH4417
customer name	R Okafor
date of hire	03/09/2026
model of bike	Electric
returned	FALSE

.....

.....

.....

2. **Pointers**. Write pseudocode to declare each of the following pointer data types. [2]

(a) **TCustomerPointer**, which points at a **STRING**

(b) **TModelPointer**, which points at the enumerated type you declared in question 1(a)

.....

3. **A set**. A theatre labels its seat rows with the capital letters **A** to **E** only. [4]

Write pseudocode to declare a set data type **RowSet**, and to define a set **ValidRows** containing those five letters.

.....

.....

4. Describe what is meant by a **non-composite** data type. Include an example in your answer. [3]

.....

.....

5. Describe a **set** data type. [3]

.....

.....

6. A student stores a club's members using two arrays: [2]

```
DECLARE MemberName : ARRAY[1:200] OF STRING
DECLARE MemberFee : ARRAY[1:200] OF REAL
```

The program then sorts **MemberFee** into ascending order. It runs without any error message.

Explain what is now wrong with the data, and state the **user-defined data type** that would have prevented it.

.....

.....

 **The Ice-Cream Challenge** — records, rebuilt so they cannot lie. (0 marks. 1 ice cream.) Open **challenge_A01_Records.py**, fill in **build_class()** and **sort_by_grade()**, and run it until every row is a ✓. **It cannot be faked**: the judge checks that the set of (name, grade) pairs is identical before and after — so if Chen's 58 ends up on Amir you fail, however well sorted the numbers are.

— **Want to dig?** In question 1(a) you refused to store the bike model as a number, because a number lets somebody write 7 and mean nothing. Now find out how an enumerated type is *actually stored*. The compiler numbers your values in the order you wrote them — **Classic** is 0, **Electric** is 1, and so on — so underneath it is an integer after all. So was the argument for nothing? No — and working out what the enumerated type still buys you, when the storage is identical, is the real lesson. (Clue: the difference is not in what the machine holds, but in what the compiler will let you type.)